

Python Fundamentals: The Survival Guide

Weeks 2–4: Logic, Functions, & Files

CMPT 211

Concordia University of Edmonton

Friday, Feb 6, 2026

The 3 Pillars of Programming

Forget the fancy terms. Every program is built from just three blocks:

1. Logic

(Week 2)



Making Decisions

(if / else)

2. Structure

(Week 3)



Grouping Tasks

(def function)

3. Safety

(Week 4)



Handling Errors

(try / except)

1. Logic (The if Block)

The Rule: Python uses **Indentation** (Space) to group code.

```
temperature = 30

# The colon (:) starts the block
if temperature > 25:
    # 4 SPACES HERE -> Inside the 'if'
    print("It is hot.")
    print("Drink water.")

# NO SPACES HERE -> Outside the 'if'
print("Check complete.")
```

If you forget the spaces, Python will crash (IndentationError).

2. Loops (The for Block)

The Rule: "For every item in this list, do this action."

```
# A list of numbers
prices = [10, 5, 20]

total = 0

# 'p' is a variable we create right here
for p in prices:
    # Inside the loop (Indented)
    print(f"Adding: {p}")
    total = total + p

print(f"Final Total: {total}")
```

3. Functions (The def Block)

Concept: A Function is a named tool.

- **Input:** What the tool needs (Parameters).
- **Output:** What the tool gives back (Return).

```
# DEFINITION (Building the tool)
def check_password(input_text):
    if input_text == "secret123":
        return True
    else:
        return False

# EXECUTION (Using the tool)
is_allowed = check_password("12345") # Returns False
```

The Most Common Mistake

Print vs. Return

Print puts text on the screen for the human.

Return sends data back to the computer code.

Wrong (Dead End)

```
def add(a, b):  
    print(a + b)  
  
x = add(2, 2)  
# x is Empty (None)!
```

Correct (Chainable)

```
def add(a, b):  
    return a + b  
  
x = add(2, 2)  
# x is 4
```

4. Safety (The try Block)

Programs crash when data is bad. We use try to catch the crash.

```
1 user_input = "hello"
2
3 try:
4     # 1. Try the dangerous code
5     number = int(user_input)
6     print("Success!")
7
8 except ValueError:
9     # 2. Run this if step 1 fails
10    print("Error: That is not a number.")
11
12 print("I am still running!")
```

5. Files (The with open Block)

How to save data so it lasts after the program stops.

```
message = "Log Entry: System Started"

# "w" = Write Mode (Create new file)
# "a" = Append Mode (Add to end)

try:
    with open("log.txt", "w") as f:
        f.write(message)
        print("Saved successfully.")

except Exception:
    print("Could not save file.")
```


Putting It All Together

A "Simple Calculator" using all 3 weeks of topics.

```
def save_result(result):  
    with open("results.txt", "a") as f:  
        f.write(f"Result: {result}\n")  
  
def divide(a, b):  
    try:  
        answer = a / b  
        return answer  
    except ZeroDivisionError:  
        return "Error: Cannot divide by zero"  
  
# Main Program  
val = divide(10, 2)  
if val != "Error":
```

The Safety Net: Try & Except

Why do we need this?

- Normally, if an error happens, the program **crashes** (stops immediately).
- We want the program to **survive** errors (like a user typing text instead of numbers).

The Logic Flow:

- **try:** "Attempt to run this dangerous code."
- **...If it works:** Skip the 'except' block and continue.
- **...If it fails:** Stop immediately and jump to the 'except' block.
- **except:** "Run this emergency plan so we don't crash."

What is a JSON file?

Context: We don't want to hard-code settings (like passwords) inside our Python script. We save them in a separate text file called **JSON**.

The Anatomy:

- { }: Curly Braces act as the container.
- "Key":: The name of the setting. *Must use double quotes!*
- **Value**: The actual data (Number, String, True/False).

Example (config.json)

```
1 {  
2     "app_name": "  
        StudentPortal",  
3     "version": 1.5,  
4     "dark_mode": true  
5 }
```

How Python sees it:

Python reads this file and turns it into a **Dictionary**.
`settings["app_name"]` gives you `"StudentPortal"`.